quant_eval: A Behavioral Evaluation Harness for Full-Weight and Quantized Large Language Models

Capability-Resolved, Artifact-Aware Measurement of Agent-Relevant Behavior
(Version 7.21)

Patrick Hill, M.S.
Founder & Principal AI/ML Systems Architect
PBH Applied Systems, LLC

July 2026

Abstract

Perplexity scores and static leaderboards answer the wrong question for practitioners deploying language models as agents: they measure aggregate language-modeling quality, not whether a specific model — at a specific precision, on specific hardware — can reliably emit valid structured output, dispatch the correct tool with the correct arguments, or carry state across turns. This gap widens under quantization, where a 4-bit checkpoint may preserve some agent-relevant behaviors while silently losing others. This whitepaper documents quant_eval version 7.21, a proprietary behavioral evaluation harness developed by PBH Applied Systems, LLC that measures agent-relevant behavior directly and resolves it per capability. The framework evaluates full-weight (16-bit) and quantized checkpoints — GGUF via llama.cpp, NormalFloat-4 via bitsandbytes, and W4A16 compressed-tensors via vLLM — through one backend-agnostic, adapter-isolated pipeline, scoring each model against a versioned, hash-pinned set of golden-oracle fixtures across eight test families. Its central methodological commitment is the separation of harness artifacts (scoring failures caused by output-format drift rather than model weakness) from genuine capability degradation, with genuine findings confirmed through re-run stability. We ground the description in a worked example: the Mistral-Nemo-Instruct-2407 model evaluated at three precisions through the same harness. A side-by-side 16-bit-versus-Q4_K_M run exposes a per-capability degradation profile — tool-call formatting, state tracking, and single-step structured output are preserved, while multi-step planning and multiple-choice accuracy degrade — alongside one tool-call scoring failure that the framework's centralized normalization identifies as an artifact and a later run on a second backend independently corroborates. The result is not a single quality number but a capability profile that functions as a selection layer for agent construction: which model, on which hardware, for which behavior.

Keywords: large language models, quantization, behavioral evaluation, agentic AI, tool calling, structured output, model selection, reproducibility

Contents

1. Introduction

1.1 The Deployment Gap

Open-weight language models are typically selected using two kinds of evidence: aggregate benchmark leaderboards and, for local deployment, the availability of a small quantized checkpoint that fits on consumer hardware. Neither kind of evidence answers the question a practitioner building an agent actually asks. Leaderboards report averaged accuracy across knowledge and reasoning benchmarks; quantization documentation reports memory footprint and, occasionally, perplexity relative to the full-weight base. A model can rank well on both

and still fail in production because it emits JSON that does not validate against a schema, names a tool under the wrong key, or loses track of state between turns. Holistic evaluation work has argued that single-number leaderboard summaries obscure the multidimensional nature of model behavior and that evaluation should be resolved across distinct capabilities and conditions rather than collapsed into one score (Liang et al., 2023). The concern is sharper for agentic deployment, where the relevant unit of success is not "answered the question" but "produced a machine-consumable artifact the surrounding system could act on."

Quantization compounds the problem. Post-training weight quantization methods reduce a model to 4-bit precision to make it deployable on modest GPUs — activation-aware weight quantization protects a small fraction of salient weight channels to limit error (Lin et al., 2024), and NormalFloat-4 provides an information-theoretically motivated 4-bit data type for normally distributed weights (Dettmers et al., 2023). These methods are validated primarily against perplexity and standard accuracy benchmarks. They do not report whether tool-argument fidelity survives, whether multi-step planning holds, or whether the model's stop semantics remain correct — precisely the behaviors on which an agent depends. In practice, quantization degrades capabilities *unevenly*: a single "how much did it degrade" number is not actionable, because the honest answer is "it depends which behavior you need."

1.2 A Behavioral, Capability-Resolved Alternative

`quant_eval` is a behavioral evaluation harness built to close this gap. It is behavioral in that every score derives from whether the model *did the task correctly* — valid JSON against a declared schema, the right tool with the right arguments, a correct multi-step plan with correct stop semantics — rather than from token-level likelihood. It is capability-resolved in that it reports a profile across eight distinct test families rather than a scalar, so a model that keeps tool calling but loses multi-step planning is described as exactly that. It is quantization-aware in that its native mode runs the full-weight (16-bit) model and the quantized model side by side on identical seeded fixtures, isolating what quantization costs *per capability*. The framework is not limited to quantized models; it evaluates full-weight checkpoints as first-class subjects. The only practical restriction is hardware: the reference evaluation node is a single NVIDIA RTX 4090 with 24 GB of VRAM, so full-weight baselines are run only when the 16-bit model fits, and larger models are evaluated quant-only with per-family pass rates carrying the signal.

The framework is at version 7.21 and supports four inference backends behind one interface: 16-bit inference via Hugging Face Transformers (Wolf et al., 2020), GGUF quantization served through llama.cpp (Gerganov & llama.cpp contributors, 2024), NormalFloat-4 via bitsandbytes (Dettmers et al., 2023), and W4A16 compressed-tensors served through the vLLM asynchronous server (Kwon et al., 2023). All four converge on the same downstream scoring pipeline and the same fixture set.

1.3 Contributions

This whitepaper makes four claims about `quant_eval` and substantiates each against real run data:

1. **Per-capability degradation measurement.** The harness runs full-weight and quantized checkpoints on identical seeded oracle fixtures and reports the difference *per behavior*, not as a single aggregate (Section 6.2).
2. **Artifact-versus-genuine discipline.** A large share of apparent "failures" in structured-output evaluation are scoring artifacts caused by output-format drift, not model weakness. The framework maintains a named taxonomy of such artifacts and centralizes their

normalization, and it confirms genuine findings through re-run stability. We demonstrate both a genuine degradation and a recovered artifact from the same run (Section 6).

3. **Backend-agnostic, adapter-isolated design.** One harness spans 16-bit, GGUF, NormalFloat-4, and compressed-tensors backends through a per-model adapter contract and a dynamic run manifest, with a distinct code path for models whose quantized artifact is not a GGUF (Section 4).
4. **Evaluation as a selection layer.** The per-model capability profile is a decision instrument for agent construction — which model, on which hardware, for which behavior — rather than a ranking (Sections 6–7).

The remainder of the paper is organized as follows. Section 2 situates the framework in the quantization and evaluation landscape. Section 3 describes the architecture. Section 4 details the methodology: the eight test families, the scoring model, the golden-oracle fixtures, and the artifact taxonomy. Section 5 presents the worked example. Section 6 reports and discusses results. Section 7 addresses limitations and reproducibility. Section 8 concludes.

2. Background and Related Work

2.1 Post-Training Quantization of Language Models

Quantization reduces the numerical precision of model weights to shrink memory footprint and increase inference throughput. For a 12-billion-parameter model such as the one used in this paper's worked example, 16-bit weights occupy roughly 24 GB, placing the full-weight model at the ceiling of a 24 GB consumer GPU; a 4-bit quantization reduces the same weights to under 8 GB, comfortably within reach of an 8–12 GB card. The question `quant_eval` exists to answer is what that reduction costs behaviorally, and the answer depends on both the quantization algorithm and the serving stack. The framework treats several distinct 4-bit regimes as separate subjects of measurement.

GGUF (llama.cpp). GGUF is the model container format used by llama.cpp, a C/C++ inference engine widely used for local deployment (Gerganov & llama.cpp contributors, 2024). A 16-bit Hugging Face checkpoint is converted to an F16 GGUF and then quantized with `llama-quantize` to a mixed 4-bit scheme (Q4_K_M). GGUF is the framework's baseline quantized backend and the most common local-deployment target.

NormalFloat-4 (bitsandbytes). NormalFloat-4 (NF4) is a 4-bit data type designed to be information-theoretically optimal for normally distributed weights, paired with double quantization of the quantization constants (Dettmers et al., 2023). It is applied at load time through the bitsandbytes library and served through the Transformers stack; the quantized checkpoint is an on-disk Hugging Face directory rather than a GGUF.

AWQ / compressed-tensors W4A16 (vLLM). Activation-aware weight quantization identifies salient weight channels by reference to the activation distribution and scales them to reduce quantization error without backpropagation or reconstruction, generalizing well to instruction-tuned models (Lin et al., 2024). The resulting weights can be packed in the compressed-tensors format and served through vLLM, whose PagedAttention memory manager provides high-throughput serving (Kwon et al., 2023). The scheme used in this paper's second worked-example backend is W4A16-asymmetric: 4-bit weights with 16-bit activations and per-group zero-points. Related post-training methods such as GPTQ pursue the same 4-bit goal through layer-wise reconstruction (Frantar et al., 2023); the framework's design accommodates additional schemes through the same adapter mechanism.

These regimes differ not only in algorithm but in decoding behavior — for example, in how end-of-sequence and role-delimiter tokens surface in decoded text — and those differences are themselves a source of scoring artifacts, as Section 4.5 details.

2.2 Evaluation: Perplexity, Leaderboards, and Behavior

The dominant public signals for model quality are perplexity and accuracy leaderboards. Perplexity measures how well a model predicts held-out text; it is a language-modeling proxy, not a measure of whether a model can execute a structured task. Accuracy leaderboards aggregate performance across knowledge and reasoning benchmarks into rankings. Holistic evaluation research has argued that collapsing model behavior into single numbers hides the conditions under which models succeed or fail and that evaluation should be multi-metric and multi-scenario (Liang et al., 2023). For quantized models specifically, published evaluations typically report perplexity deltas relative to the full-weight base, which does not decompose into the agent-relevant capabilities a deployment depends on.

`quant_eval` adopts the multidimensional stance but narrows the scope deliberately: rather than broad knowledge coverage, it measures a compact set of *agent-relevant* behaviors under deterministic conditions, and it measures them comparably across precisions. This is a different instrument from a leaderboard — it is designed to inform a deployment decision for one model on one hardware target, not to rank the field.

2.3 Agent-Relevant Behaviors

The behaviors the framework measures are those an agent runtime depends on. Tool use — emitting a structured call that names a function and supplies its arguments — is central to agent frameworks; the ReAct paradigm interleaves reasoning traces with such action calls (Yao et al., 2023), and models are increasingly trained for function calling directly, as the base model in this paper's worked example was (Mistral AI, 2024). Structured output means emitting JSON that validates against a declared schema; the framework validates against JSON Schema (Wright et al., 2022). Multi-step planning means producing a correct sequence of decisions with correct termination. State tracking means carrying information correctly across conversational turns. Each of these is scored as its own family, because each can degrade independently under quantization.

2.4 The Adapter Pattern as an Architectural Choice

The framework's per-model design uses the adapter pattern in its classical software-engineering sense — an object that converts one interface into another so that otherwise incompatible components can collaborate (Gamma et al., 1994). Here, each model family is wrapped in an adapter that presents a uniform contract to the pipeline while encapsulating model-specific routing and runner construction. This is an architectural decision with methodological consequences: it isolates model-specific quirks (tokenizer settings, think-block handling, quantization scheme) behind a stable boundary, so that the scoring pipeline remains identical across models and precisions. Section 3 develops this design.

3. Framework Architecture

`quant_eval` is organized as a pipeline of loosely coupled stages: model intake and adapter selection, runner construction, execution-only batteries, evaluator-owned scoring, an artifact-and-provenance layer, and report generation. The two figures below present the architecture in full. Figure 1 covers intake, adapter orchestration, and runner construction; Figure 2 covers battery execution, scoring, artifact provenance, and catalog outputs.

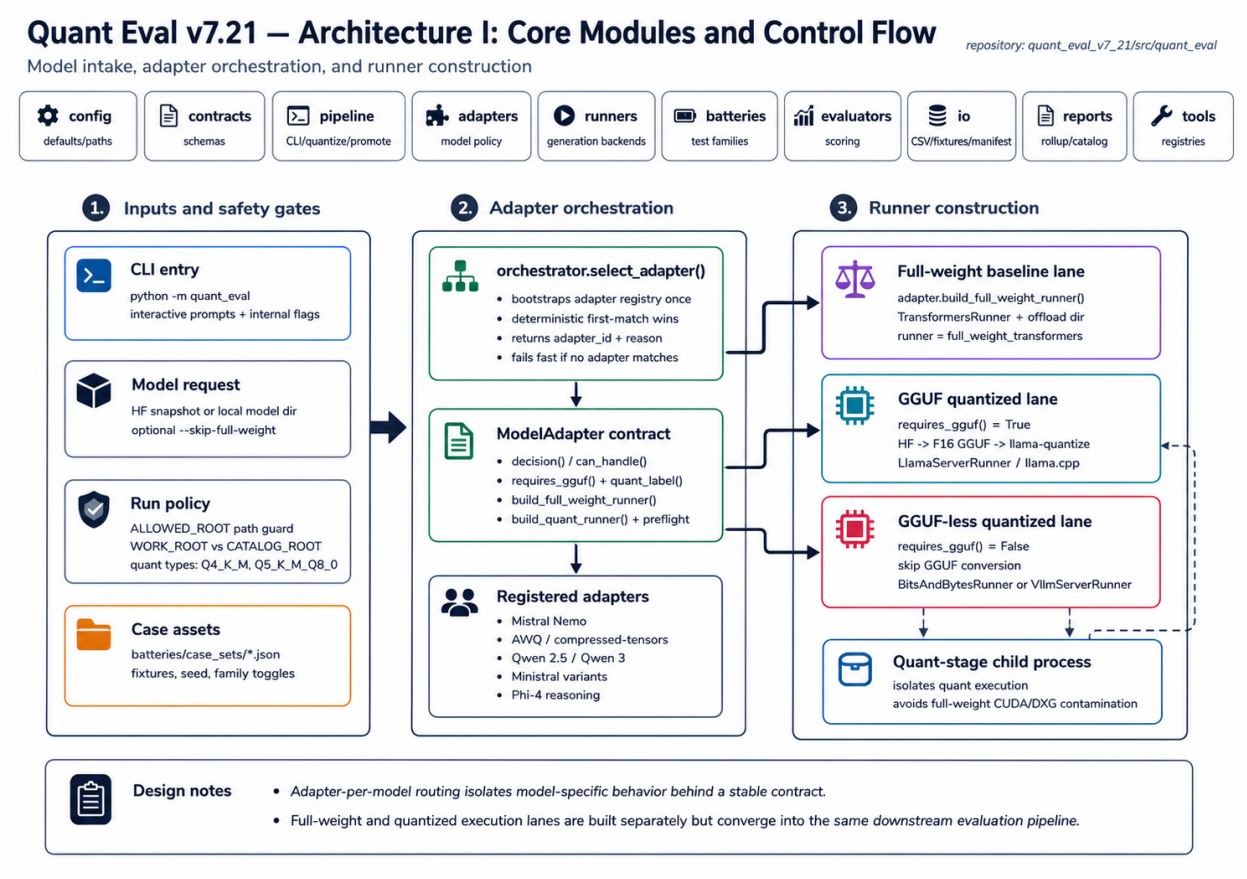

Figure 1: Architecture I: core modules and control flow — model intake, adapter orchestration, and runner construction.

3.1 Adapter-Per-Model Routing and the Registry

The central design decision is one adapter per model family. Each adapter implements a fixed contract: it declares an `adapter_id`, answers whether it `can_handle` a given model identifier or on-disk directory, returns a routing `decision` with a human-readable reason, reports capability flags (`requires_gguf`, `quant_label`), and constructs the full-weight and quantized runners (`build_full_weight_runner`, `build_quant_runner`) with a `preflight_quant_runner` sanity check. Adapters are registered in an explicit registry whose insertion order is the resolution policy: the orchestrator selects the **first** adapter whose `decision` is non-null. This makes routing deterministic and auditable rather than implicit.

Registration order carries real weight. Specialized quantization adapters are registered *before*

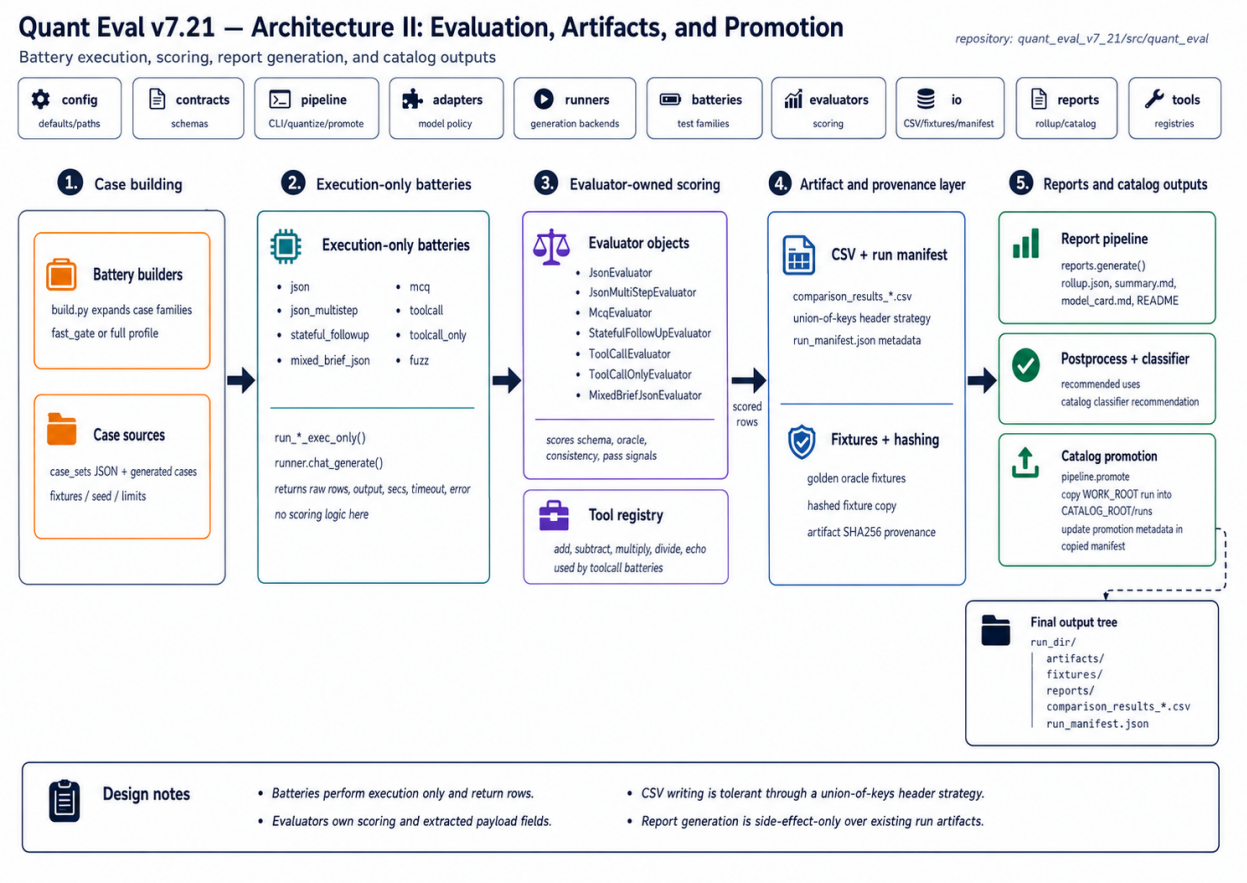

Figure 2: Architecture II: evaluation, artifacts, and promotion — battery execution, scoring, report generation, and catalog outputs.

the generic base-model adapter for the same family, and the generic adapter additionally excludes quantization markers (`awq`, `compressed`, `w4a16`, `4bit`, `bnb`) from its own matching. This belt-and-suspenders arrangement resolves a genuine collision: a directory named `Qwen2.5-32B-Instruct_4bit_...` contains the substring the generic 32B adapter would otherwise claim, so the 4-bit adapter must win first. The orchestrator fails fast with a clear message if no adapter matches, rather than guessing.

3.2 Runners: One Interface, Four Backends

Runners encapsulate inference. All expose the same `chat_generate` method — user prompt in, decoded text out, under a shared system message and a caller-supplied token budget, temperature, stop sequences, and seed. Four runners implement it:

- `TransformersRunner` performs 16-bit inference through Hugging Face Transformers (Wolf et al., 2020). It is the full-weight baseline lane. It loads to a single GPU when the model fits and falls back to CPU offload with a memory cap when it does not.
- `LlamaServerRunner` serves a GGUF quantization through a llama.cpp server subprocess.
- `BitsAndBytesRunner` serves an on-disk NormalFloat-4 checkpoint. It subclasses `TransformersRunner` to reuse its generation path but overrides loading, because a pre-quantized 4-bit checkpoint cannot be dtype-cast or device-moved the way a 16-bit model can; it loads with the baked-in quantization configuration and places inputs on the embedding device manually.
- `VllmServerRunner` serves a compressed-tensors W4A16 checkpoint by spawning a vLLM OpenAI-compatible server (Kwon et al., 2023), waiting on its health and model endpoints, and answering generation requests over HTTP.

Because every runner presents `chat_generate`, the batteries that drive them are backend-agnostic; the runner's identity is recorded in the manifest and in every output row, but the execution and scoring code does not branch on it.

3.3 Two Quantization Lanes: GGUF and GGUF-Less

The pipeline routes on a single capability flag, `requires_gguf()`. When true, the model takes the **GGUF lane**: a 16-bit checkpoint is converted to an F16 GGUF, quantized with `llama-quantize`, and served by a llama.cpp runner. When false, the model takes the **GGUF-less lane**: the pre-quantized on-disk directory *is* the artifact under test, so conversion and quantization are skipped entirely and the quantized runner is built directly from the directory. NormalFloat-4 and compressed-tensors checkpoints take this lane. Everything downstream of runner construction — battery execution, per-case rows, scoring, manifest update, reports — is shared between the two lanes without modification. The quantized stage runs in a child process so that quantized execution is isolated from any residual full-weight GPU state.

3.4 Batteries and Evaluators: Separation of Execution and Scoring

A deliberate separation runs through the design: **batteries execute, evaluators score**. A battery drives a runner over a family's cases and returns raw rows — output text, wall-clock seconds, timeout and error flags — with no scoring logic. An evaluator object then consumes each raw row and produces the correctness signals for that family (schema validity, tool-name correctness, argument correctness, plan correctness, and so on). This separation matters for credibility: the same execution can be re-scored, and the scoring logic is auditable in one place per family, independent of how the output was produced. Normalization helpers that both scoring and tool execution rely on — extraction of the first balanced JSON object,

tool-name alias resolution, and argument-key normalization — live in a shared module so that a fix applies uniformly rather than per adapter (Section 4.5).

3.5 Domain, Fixtures, and Provenance

Several families are grounded in a small deterministic planning domain — a shelf-placement task with an oracle that computes the correct plan and final state for any case — so that "correct" is defined by construction rather than by a reference model. The full fixture set is versioned (`golden_oracle_fixtures_v7_21`), and each run copies the fixtures into the run directory under a content-addressed name carrying a SHA-256 prefix, records the full hash in the manifest, and checks for oracle drift against the committed fixtures. The manifest also records the adapter used and its reason, the runner name and role, the quantization type, artifact byte sizes and SHA-256 hashes, the seed, and the toolchain mode. This provenance layer is what makes a published capability profile independently verifiable.

3.6 Reports and Catalog Outputs

After scoring, a report stage derives a machine-readable rollup (per-runner, per-family pass rates and signal breakdowns) and human-readable artifacts: a summary, a model card, a bundle README, and a post-run checklist. A separate, strictly post-run classifier reads the rollup and derives conservative catalog recommendations — which uses the evidence supports and which it does not — without touching scoring. The report stage is side-effect-only over existing run artifacts, so regenerating reports never alters measured results.

4. Methodology

4.1 The Eight Test Families

Each model is evaluated against up to eight families of cases. The families are chosen to span the behaviors an agent runtime depends on, and each is scored by a dedicated evaluator against golden-oracle expectations.

1. `json` — single-step structured JSON output under explicit constraint rules. Gating signals: schema validity and constraint satisfaction.
2. `json_multistep` — multi-step planning with a self-check trace and oracle verification. Gating signals: schema validity, self-check consistency, correct stop semantics, and oracle equivalence of the computed final state. This is the hardest family and the most sensitive discriminator of quantization degradation.
3. `stateful_followup` — two-turn state tracking, where turn two must be correct given turn one. Gating signals: parse and exact-match on both turns.
4. `mixed_brief_json` — a hybrid response with a natural-language answer line *and* a valid JSON block. Gating signals: correct answer line, JSON parse, schema validity.
5. `mcq` — multiple-choice extraction, generated under a one-token budget. Gating signal: correct choice.
6. `toolcall` — a tool call embedded in a response, scored in two stages: stage one parses and schema-validates the call; stage two checks the final computed value.
7. `toolcall_only` — a bare, schema-only tool call. Gating signals: correct tool name and correct arguments under the tool schema.

8. `fuzz` — property-based regression over generated cases (20 by default), tracking structured-placement correctness for stability rather than as a direct agent-readiness proxy.

4.2 Determinism and the Golden Oracle

Comparability across models and across precisions requires that every model see the same inputs under the same decoding conditions. The framework fixes the seed at 42, uses greedy decoding for scoring, and applies an identical system message and identical prompts to the full-weight and quantized runs. Correctness is defined by a deterministic oracle rather than by a reference model: for the planning families, a first-fit oracle computes the correct plan, the correct per-step feasibility checks, and the correct final state for every case. Because the oracle is deterministic and the fixtures are fixed, the same case has the same correct answer for every model, and differences in score are attributable to the model, not the harness.

The fixture set is versioned and hash-pinned. Each run writes a content-addressed copy of the fixtures into the run directory and records the SHA-256 in the manifest; the runs discussed in this paper used `golden_oracle_fixtures_v7_21` with SHA-256 prefix `6d71a0b9`. A drift check compares the oracle-derived expectations against the committed fixtures and warns on any mismatch, so a silent change to the ground truth cannot pass unnoticed.

4.3 The Scoring Model

Scoring proceeds from per-case signals to per-family pass rates and, where a full-weight baseline exists, to aggregate dimension scores.

Signals are the boolean correctness checks a family's evaluator emits per case — for example, `schema_ok`, `tool_name_ok`, `args_ok`, `checks_consistent_ok`, `oracle_equiv_ok`.

Bucket score is a small per-case integer used for ranking and triage; its scale is family-specific (for example, the single-step `json` and `fuzz` families bucket to 10, and `toolcall` buckets to 11 when stage one and the final value are both correct). Bucket scores summarize a case; they are not the pass criterion.

Pass rate is the fraction of a family's cases in which *all* of that family's gating signals equal one. Pass rates are the primary catalog-facing numbers. A subtlety follows from this definition: a non-canonical output wrapper can leave `schema_ok = 0` while the behavior-relevant signals pass, so a family can report a pass rate of 1.000 with a bucket score below its maximum. This is an accepted and documented precedent, not an inconsistency — the pass rate measures whether the model did the task, and the bucket records how canonical the wrapper was.

Dimension scores are four display aggregates — coherence, instruction following, reasoning, and task completion — derived from the underlying signals. They are reported only when a full-weight baseline was run and a degradation delta is meaningful.

4.4 Side-by-Side Degradation and Quant-Only Runs

The framework's native mode runs the full-weight and quantized models on identical fixtures and reports the difference per family. This is the degradation delta: not a single number, but a per-capability comparison that shows which behaviors quantization preserved and which it did not. When the 16-bit model exceeds the 24 GB evaluation-GPU budget, the run is executed quant-only (`--skip-full-weight`): aggregate dimension scores are reported as null, and the per-family pass rates carry the signal. The two modes produce the same artifact shape; the

difference is whether a baseline column is present. Section 6 uses one of each: a side-by-side run for the degradation delta and a quant-only run on a second backend for corroboration.

4.5 The Artifact-Versus-Genuine Discipline

The most consequential methodological commitment in `quant_eval` is the separation of *harness artifacts* from *genuine capability failures*. A harness artifact is a scored failure caused by output-format drift or decoding behavior rather than by the model getting the task wrong. Treating artifacts as genuine failures would systematically understate model capability and, worse, would understate it *unevenly* across backends whose decoding conventions differ. The framework maintains a named taxonomy of recurring artifacts and neutralizes each centrally.

- **Role-label leakage.** A leading chat-template role label (for example, `assistant` or `user` followed by a newline) can survive decoding as literal text and corrupt answer-line extraction and one-token multiple-choice scoring even when the underlying answer is correct. The Transformers-path runner adds the generation prompt so the model emits assistant content directly, and strips a single leading role-label line as defense in depth.
- **End-of-sequence contamination.** Some backends can leak an end-of-sequence marker into decoded text, which can fail a stage-two final-value check; other backends decode cleanly. The framework treats decode cleanliness as backend-dependent and does not attribute a leaked marker to model weakness.
- **Tool-argument key drift.** Models and serving stacks disagree on where call arguments live — `args`, `arguments`, `params`, or `parameters` — and on operand naming (for example, `x`/`y` versus `a`/`b`). A shared helper normalizes these so scoring reflects argument *correctness*, not field naming.
- **Tool-name key drift.** Models name the called tool under different keys — `tool_name`, `tool`, `name`, `action` (the ReAct convention; Yao et al., 2023), or `function` (the OpenAI tool-use convention). A shared helper resolves these aliases so scoring reflects that the model named the right tool.
- **One-token budget under reasoning models.** The multiple-choice family generates under a one-token budget. A reasoning-style model that emits a thinking preamble cannot fit a choice into one token; a zero there is an architectural constraint of the measurement, documented as such, not scored as a capability failure.

Crucially, artifact normalization is centralized in the shared evaluator module, so a single fix applies to every family and every backend rather than being reimplemented per adapter. Genuine findings are distinguished from artifacts by *re-run stability*: a failure that persists across re-runs and across backends is a genuine capability result, whereas a failure that a correct normalization removes — and that a second backend does not reproduce — is an artifact. Section 6 exhibits one of each from real runs.

4.6 Evaluation Profiles

To control runtime cost without changing scoring, the framework supports two profiles. `fast_gate` truncates each family to a small case cap for quick screening; `full` emits the complete case lists. Profiles change only how many cases a family emits, never how a case is scored. The runs in this paper were executed under `fast_gate`, which caps the families at the counts reported in Section 6 (for example, five `json_multistep` cases and five `mcq` cases). This has a direct consequence for interpretation, addressed in Section 7: with fewer than ten cases per family, readiness verdicts are marked provisional, and a single case moves a pass rate substantially.

5. Worked Example: One Model, Three Backends

To make the methodology concrete, this section evaluates a single base model — Mistral-Nemo-Instruct-2407, a 12-billion-parameter instruction-tuned model jointly released by Mistral AI and NVIDIA under Apache 2.0, trained for function calling and a 128K context window (Mistral AI, 2024) — through the same harness at three precisions:

- **FP16** via `TransformersRunner` (the full-weight baseline),
- **Q4_K_M GGUF** via `LlamaServerRunner` (the baseline quantized backend), and
- **W4A16 compressed-tensors (AWQ)** via `VllmServerRunner` (the GGUF-less backend).

The first two were executed as a single **side-by-side run** (run identifier `20260211_022944`), producing a per-capability degradation delta. The third was a later **quant-only run** on a different backend (run identifier `20260629_184555`), used here to corroborate the artifact analysis. Both runs used the same fixture set (`golden_oracle_fixtures_v7_21`, SHA-256 prefix `6d71a0b9`), seed 42, and the `fast_gate` profile, on an RTX 4090.

5.1 The Degradation Delta (FP16 vs. Q4_K_M)

Table 1 reports the per-family result for the side-by-side run. Pass rate is the gating-signal conjunction (Section 4.3); for the two families reported by bucket average (`json` and `fuzz`), the average bucket is shown.

Table 1: Per-family results, Mistral-Nemo-Instruct-2407, FP16 vs. Q4_K_M (run `20260211_022944`).

Family	Metric	FP16	Q4_K_M	Change	Interpretation
`json`	avg bucket	10.000	10.000	0	Preserved
`fuzz`	avg bucket	10.000	10.000	0	Preserved
`stateful_followup`	pass rate	1.000	1.000	0	Preserved
`mixed_brief_json`	pass rate	1.000	1.000	0	Preserved
`toolcall`	pass rate (stage 1)	1.000	1.000	0	Stage-1 preserved
`toolcall`	avg bucket	11.000	5.500	−5.500	Final-value degraded (§5.2)
`json_multistep`	pass rate	0.600	0.400	−0.200	Genuine degradation (§5.2)
`mcq`	avg bucket (of 1)	0.600	0.400	−0.200	Genuine degradation (§5.2)
`toolcall_only`	pass rate	1.000	0.000	−1.000	Harness artifact (§5.3)

FP16 = full-weight Transformers runner; Q4_K_M = llama.cpp runner. Change is Q4_K_M minus FP16.

The shape of the result is the point. Quantization to Q4_K_M **preserved** single-step structured JSON, property-based regression, two-turn state tracking, the hybrid brief-plus-JSON format, and stage-one tool-call formatting. It **genuinely degraded** multi-step planning, multiple-choice accuracy, and tool-call final-value computation. And it produced **one apparent collapse** — bare tool-call dispatch falling from 1.000 to 0.000 — that the framework identifies as an artifact rather than a capability loss. A single aggregate "degradation" number would blur all three categories together; the capability-resolved profile keeps them distinct, which is what a deployment decision requires.

5.2 Genuine Degradations

Multi-step planning (`json_multistep`, 0.600 → 0.400). The signal breakdown localizes the loss. Schema validity fell from 1.000 to 0.800 and self-check consistency from 0.800 to 0.400, while oracle equivalence held at 0.600. Under Q4_K_M, one hard case collapsed entirely — the model produced no parseable plan where the full-weight model had produced a (partially correct) one — and the model's intermediate self-check reasoning became less internally consistent. This is a genuine and unsurprising result: multi-step planning with a self-check trace is the most fragile family, and it is the first to erode under 4-bit quantization. The deployment implication is explicit in the model card derived from this run: do not rely on this checkpoint for unassisted multi-step planning; wrap it in an external planner or validation loop.

Multiple-choice (`mcq`, 3/5 → 2/5). Extraction was clean at both precisions (single-letter outputs, no parsing artifact). One item that the full-weight model answered correctly (`B`) flipped to an incorrect `A` under Q4_K_M. This is a genuine capability result — a mild accuracy loss on a harder item — not a scoring problem.

Tool-call final value (`toolcall`, bucket 11.000 → 5.500). Both `toolcall` cases still passed stage one at Q4_K_M: the tool-call JSON parsed and validated. But on one of the two cases the model's *final computed value* was wrong — it emitted `123` where the correct sum of `10` and `-4` is `6` — so the stage-two value check failed and that case's bucket dropped from 11 to 0, halving the family average. The stage-one formatting behavior was preserved; the arithmetic in the free-text final answer was not. Reporting these two stages separately is what makes the degradation legible: "tool calling" did not simply "break," it retained correct call structure while losing final-value reliability.

5.3 An Artifact, Confirmed Two Ways

The most instructive line in Table 1 is `toolcall_only`, which fell from a perfect 1.000 to 0.000. Taken at face value, this says the quantized model lost the ability to call a tool. The raw outputs say otherwise. The tool name was correct in every case (`tool_name_ok = 1.000` at both precisions); the failure was entirely in `args_ok`. The full-weight model emitted its arguments under an `args` wrapper (and, in one case, under `x`/`y` operand names), which the framework's normalization maps to the canonical form, yielding correct arguments. The quantized model emitted its arguments under a `params` wrapper — a recognized alias — but with a trailing extra brace, producing output such as `{"tool": "add", "params": {"a": 5, "b": 10}}`.

Two independent lines of evidence establish that this is a harness artifact, not a capability loss:

1. **Centralized normalization recovers it.** Re-scoring the exact quantized outputs through the version 7.21 shared normalization helpers extracts the first balanced JSON object (discarding the trailing brace), resolves the `params` alias, and returns the correct arguments `{"a": 5, "b": 10}` and `{"a": 25, "b": 75}`. The behavior the family is meant to measure — did the model name the right tool with the right operands — succeeded; the 0.000 reflects wrapper-and-punctuation drift that the centralized fix neutralizes.
2. **A second backend does not reproduce it.** The later quant-only AWQ run on vLLM scored `toolcall_only` at 1.000 (Table 2). In that run the model free-formed to the ReAct `action` key with `x`/`y` operands, which normalization again resolves. If the 0.000 were a genuine quantization-induced capability loss, a second, independently quantized checkpoint of the same model would be expected to show impairment; it does not.

This is the artifact-versus-genuine discipline operating exactly as intended: a scored failure was quarantined as a harness effect, corroborated by re-run stability across a second backend,

and prevented from understating the model's true tool-calling capability in the catalog.

5.4 Backend-Agnostic Corroboration (W4A16 AWQ)

Table 2 reports the quant-only AWQ run, served through vLLM on the GGUF-less lane. It is not a degradation delta — no FP16 baseline is attached — but it confirms the capability profile through a completely different quantization algorithm and serving stack.

Table 2: Per-family results, Mistral-Nemo-Instruct-2407 AWQ W4A16 (quant-only, run `20260629_184555`).

Family	N	Pass rate	Avg bucket	Notes
`json`	4	n/a	10.000	All pass
`fuzz`	20	n/a	10.000	All pass
`stateful_followup`	2	1.000	2.000	Preserved
`mixed_brief_json`	2	1.000	2.000	Preserved
`toolcall`	2	1.000	11.000	Stage-1 and final correct; clean decode
`toolcall_only`	2	1.000	2.000	`action` alias normalized
`json_multistep`	5	0.400	1.800	Same fragile family
`mcq`	5	n/a	0.600	3/5

: Per-family results, Mistral-Nemo-Instruct-2407 AWQ W4A16 (quant-only, run `20260629_184555`).

The AWQ profile agrees with the Q4_K_M profile on the preserved families and on the fragile one (`json_multistep` at 0.400, `mcq` at 3/5). It differs on exactly the two lines the artifact analysis predicted: `toolcall_only` is 1.000 (the argument-key drift is normalized rather than tripping on a trailing brace), and `toolcall` recovers a clean bucket of 11.000 (the vLLM path decoded the final value without contamination). One base model, two quantization algorithms, two serving stacks, one harness — and a consistent capability story.

5.5 Cost and Footprint

The behavioral profile is only half of a deployment decision; the other half is cost. The run manifest records the artifact sizes directly: the F16 GGUF was 24,504,279,808 bytes (~24.5 GB) and the Q4_K_M GGUF was 7,477,207,808 bytes (~7.48 GB), a roughly 3.3× reduction that moves the model from the ceiling of a 24 GB GPU to an 8 GB one. Wall-clock cost moved commensurately: mean per-case latency was 30.24 s under FP16 Transformers versus 1.42 s under Q4_K_M llama.cpp — a roughly 21× speedup — and 0.936 s under AWQ on vLLM. These figures are eval-harness measurements under a fixed configuration, not production throughput benchmarks, but they quantify the trade the capability profile is set against: for this model, the preserved families are available at a fraction of the footprint and latency, and the price is paid specifically in multi-step planning and multiple-choice reliability.

6. Discussion

6.1 Evaluation as a Selection Layer

The output of a `quant_eval` run is not a rank; it is a per-model, per-backend capability profile with honest cautions. For the worked example, that profile reads: deploy with confidence for tool dispatch, two-turn stateful interaction, hybrid brief-plus-JSON output, and single-step structured JSON; deploy with an external validation loop for multi-step planning; verify on your own option distributions before relying on multiple-choice; and, in a serving stack that does not normalize tool wrappers, enforce the canonical tool-name and argument keys through the system prompt. That is a set of engineering instructions, not a leaderboard position. When such profiles are produced for many models across the backends a team can actually run, they compose into a selection layer: given a required behavior and a hardware budget, the profiles narrow the field to the checkpoints that measurably support that behavior at that budget.

This reframing is the practical payoff of the capability-resolved and artifact-aware commitments. A single degradation number would have told a team only that Q4_K_M "loses a little" relative to FP16. The profile tells them precisely *what* it loses (multi-step planning consistency, one multiple-choice item, tool-call final-value reliability), *what it keeps* (formatting, state, single-step structure), and *what looked lost but was not* (bare tool dispatch). Only the last of these is visible without the artifact discipline, and it is exactly the kind of false negative that would otherwise cause a usable model to be wrongly excluded from a catalog.

6.2 Why the Artifact Discipline Is the Load-Bearing Claim

Across the worked example, the difference between a defensible profile and a misleading one came down to three scoring failures that had to be classified correctly. Two were genuine (`json_multistep`, `mcq`, and the `toolcall` final value) and were confirmed by their persistence across re-runs and, for the fragile planning family, across both quantized backends. One was an artifact (`toolcall_only`) and was confirmed by centralized normalization recovering the correct behavior and by a second backend declining to reproduce the failure. Had the framework scored the artifact as genuine, it would have published that a function-calling model cannot call functions after quantization — a conclusion contradicted by the model's own correct tool name and correct operands. The credibility of the entire catalog rests on getting this classification right, which is why normalization is centralized in one shared module rather than scattered across adapters, and why genuine findings are held to a re-run-stability standard before they are reported.

This is also why the framework deliberately avoids product or persona models. A model shipped with a baked-in default system prompt fights the harness's deterministic system-message injection, which both contaminates scoring and defeats the comparability that makes a degradation delta meaningful. Selecting instruction-following and tool-use models rather than product models is a methodological prerequisite, not a preference.

6.3 What the Numbers Do and Do Not Support

The worked example supports strong claims about the *framework* — that it measures per-capability degradation, that it separates artifacts from genuine failures, and that it does so identically across three backends — because those claims are about the instrument and are demonstrated by the instrument's behavior on real runs. It supports only provisional claims about the *model*, because the runs were executed under the `fast_gate` profile with small per-family case counts. A pass rate of 0.400 on five `json_multistep` cases means two of five

passed; a single additional passing case would move it to 0.600. The framework marks every such verdict provisional (with an explicit "n < 10" annotation) precisely so that a screening result is not mistaken for a definitive capability measurement. The distinction matters: this paper's contribution is the evaluation methodology and its demonstrated behavior, and the specific pass rates are illustrative of the method rather than final grades for the model.

6.4 Generality Across Backends and Models

The same adapter contract, the same fixtures, and the same scoring pipeline have been exercised across the framework's supported backends — 16-bit Transformers, GGUF, NormalFloat-4, and compressed-tensors W4A16 — and across multiple model families beyond the worked example. The GGUF-less lane, added to accommodate quantized artifacts that are on-disk directories rather than GGUF files, reuses the entire downstream path unchanged; the only branch is whether a conversion-and-quantize step runs before the shared execution and scoring. This is the concrete meaning of "backend-agnostic": the cost of adding a new quantization regime is a runner and an adapter, not a parallel evaluation stack, and the resulting scores are directly comparable to every prior run because they flow through the same fixtures and the same evaluators.

7. Limitations and Reproducibility

7.1 Limitations

Small case counts under `fast_gate`. The runs in this paper used the screening profile, with per-family counts as low as two and at most twenty. Pass rates on such small samples are coarse, and the framework reports readiness verdicts as provisional when a family has fewer than ten cases. Screening results identify candidates and localize likely weaknesses; they are not a substitute for a full-profile run before a production commitment.

Hardware-bounded baselines. The reference evaluation node is a single 24 GB GPU. Models whose 16-bit weights exceed that budget cannot be run full-weight on this node and are evaluated quant-only, which yields per-family pass rates but no degradation delta. This is a property of the current evaluation hardware, not of the framework, which evaluates full-weight and quantized checkpoints identically when both fit.

Compact, domain-anchored fixtures. Several families are grounded in a single deterministic planning domain. This buys an unambiguous oracle and exact comparability at the cost of breadth: the fixtures probe agent-relevant *behaviors* precisely rather than broad world knowledge. The framework is a behavioral instrument, not a general knowledge benchmark, and should be read as such.

Single-model worked example. This paper demonstrates the methodology on one base model across three backends. That design is deliberate — it isolates backend and precision effects while holding the model constant — but it means the specific numbers characterize one model, and the framework's cross-model generality is asserted from its architecture and its broader catalog rather than proved in this document.

Greedy, seeded decoding. Scoring uses greedy decoding at a fixed seed for determinism and comparability. This measures the model's most-likely behavior under fixed conditions; it does not characterize sampling-temperature behavior or best-of-k variability beyond the framework's optional selection mechanisms.

7.2 Reproducibility

Every run is designed to be independently verifiable from its recorded artifacts.

Fixtures. The exact test cases are versioned (`golden_oracle_fixtures_v7_21`) and copied into the run directory under a content-addressed filename carrying a SHA-256 prefix; the full hash (`6d71a0b9147c...`) is recorded in the manifest, and an oracle-drift check guards against silent changes to the ground truth.

Manifest. `run_manifest.json` records the adapter and its selection reason, the runner name and role, the quantization type and toolchain mode, the seed (42), and — for the GGUF run — the byte sizes and SHA-256 hashes of the F16 and Q4_K_M artifacts and the SHA-256 of the conversion script and quantization binary.

Per-case CSV. The full per-case results are published as a comparison CSV (one row per case per runner) containing the raw output, timing, and every scored signal, so that any pass rate or bucket score can be recomputed and any individual failure inspected at the row level.

Derived reports. The machine-readable rollup, the human-readable summary and model card, the bundle README, and the post-run checklist are all regenerable from the CSV and manifest without re-running inference, and the catalog classifier that derives recommended uses is a strictly post-run, non-invasive read of the rollup.

Together these artifacts allow a third party to recompute the reported pass rates from the CSV, verify the model artifacts against the recorded hashes, and re-derive the reports — the standard of reproducibility the framework's credibility depends on.

8. Conclusion

Perplexity and static leaderboards do not tell a practitioner whether a given model — full-weight or quantized, on the hardware they actually have — can run an agent. `quant_eval` version 7.21 answers that question directly by measuring agent-relevant behavior, resolving it per capability, and reporting the result as a profile rather than a score. Its native mode runs the full-weight and quantized checkpoints side by side on identical seeded oracle fixtures and reports the degradation *per behavior*; its GGUF-less lane extends the same measurement to quantized artifacts that are not GGUF files; and its artifact-versus-genuine discipline — centralized normalization plus re-run-stability confirmation — keeps scoring failures from being misread as capability losses.

The worked example makes the case in miniature. One base model at three precisions, through one harness, produced a capability profile in which single-step structure, state tracking, and tool-call formatting were preserved under 4-bit quantization; multi-step planning, multiple-choice accuracy, and tool-call final-value reliability genuinely degraded; and one apparent collapse of bare tool dispatch was correctly quarantined as a harness artifact — confirmed both by centralized normalization recovering the correct behavior and by a second, independently quantized backend declining to reproduce the failure. That last classification is the framework's most load-bearing move: without it, a function-calling model would have been recorded as unable to call functions after quantization, a false negative that the artifact discipline prevents.

The result is an evaluation instrument that functions as a selection layer for agent construction. Given a required behavior and a hardware budget, a catalog of such profiles narrows the field to the checkpoints that measurably support that behavior at that budget, with the specific guardrails each one needs. The contribution is not a benchmark number but a disciplined, reproducible, backend-agnostic method for producing the evidence a deployment decision actually requires.

References

Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. *Advances in Neural Information Processing Systems, 36*, 10088–10115.

Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2023). GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*. https://openreview.net/forum?id=tcbBPnfwxS

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Gerganov, G., & llama.cpp contributors. (2024). *llama.cpp* [Computer software]. GitHub. https://github.com/ggml-org/llama.cpp

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)* (pp. 611–626). Association for Computing Machinery. https://doi.org/10.1145/3600006.3613165

Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., Zhang, Y., Narayanan, D., Wu, Y., Kumar, A., Newman, B., Yuan, B., Yan, B., Zhang, C., Cosgrove, C., Manning, C. D., Ré, C., Acosta-Navas, D., Hudson, D. A., … Koreeda, Y. (2023). Holistic evaluation of language models. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=iO4LZibEqW

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., & Han, S. (2024). AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. *Proceedings of Machine Learning and Systems, 6*, 87–100.

Mistral AI. (2024, July 18). *Mistral NeMo*. https://mistral.ai/news/mistral-nemo/

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., … Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (pp. 38–45). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-demos.6

Wright, A., Andrews, H., Hutton, B., & Dennis, G. (2022). *JSON Schema: A media type for describing JSON documents* (2020-12 Internet-Draft). Internet Engineering Task Force. https://json-schema.org/

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*. https://openreview.net/forum?id=WE_vluYUL-X